

Generative 3D Object Modeling for Robust Robot Manipulation in ROS 2

Abdulrahman S. Al-Batati¹ and Yasser Alhabashi¹ and Khaled Gabr¹ and Serry Sibae¹
and Mohamed Abdelkader¹ and Wadii Boulila¹

Abstract—

Simulation plays a central role in robotic manipulation development. Simulation drives modern robotic manipulation research, yet the diversity of available 3D object assets is often limited to small, biased CAD collections, which can reduce the robustness of motion planning and learning pipelines. This paper presents a dataset-free framework for generative 3D object modeling tailored to targeted manipulation in ROS2. Our approach represents objects as compositions of simple 2D [1]. Each candidate object is expressed as a union of parametric primitives and assigns each candidate a robotics-driven score based on size feasibility, static stability proxies, mesh validity, and a continuous graspability metric. To avoid the need for external datasets, we employ a lightweight cross-entropy optimization scheme that iteratively, and mesh validity. A lightweight Cross-Entropy optimizer [2] adapts the parameter distribution toward high-scoring objects. The resulting generator outputs meshes together with physically without any external dataset, and every accepted object is exported with consistent inertial properties and collision geometry, automatically exported to URDF/SDF and integrated into a ROS friction-sampled surfaces, ready for MoveIt 2 pipeline with MoveIt [3] and Gazebo [4]. To break the circularity that arises when training and evaluation share an objective, we report three independent downstream metrics on a Franka Panda [5]: force-closure grasp synthesis, MoveIt 2 motion-planning success, and Gazebo dynamic stability. Averaged over three seeds, our generator reaches 97.7%, 100%, and Gazebo. Large-scale experiments involving over 200,000 generated objects across 20 distinct archetypes demonstrate that our method consistently discovers stable, manipulable geometries without human intervention, significantly expanding the diversity of available simulation assets compared to fixed CAD sets. The source code is available at [archetypes](https://github.com/xya22er/3d_generator). Source code: https://github.com/xya22er/3d_generator.

I. INTRODUCTION

A. Motivation

Robotic manipulation systems increasingly rely on simulation for algorithm development, motion planning, and learning, particularly within modern ROS 2-based pipelines that combine Manipulation research increasingly leans on simulation: motion planners are tuned, grasp

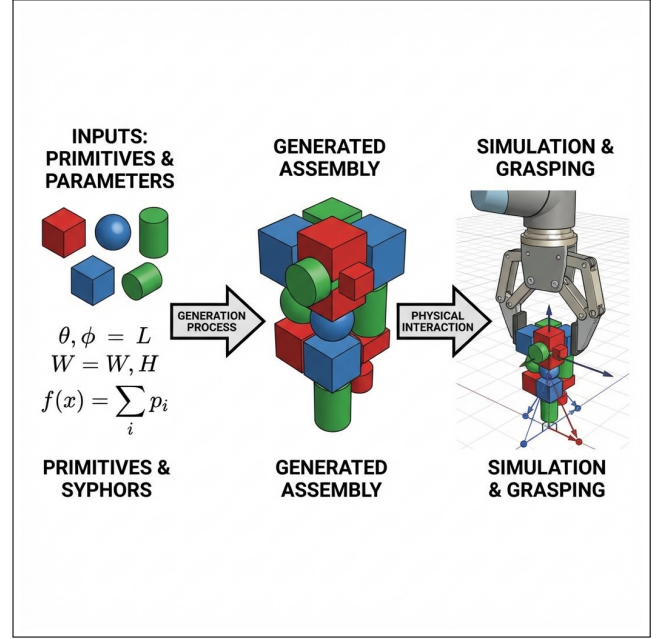


Fig. 1. Graphical Abstract: Our pipeline generates parametric primitive assemblies of primitives (left), optimizes them for stability and graspability (center), and enables direct simulation deployed for ROS 2 manipulation tasks experiments (right).

policies are learned, and benchmarks are reported from ROS 2 [1] stacks that pair physics simulators such as Gazebo with motion planning [4] with motion-planning frameworks like MoveIt 2. While simulation enables scalable experimentation, its effectiveness is strongly influenced by the diversity and realism of the simulated environments and objects. In practice, manipulation experiments are often conducted using a small number of manually designed CAD models or publicly available object libraries. These object sets are typically limited in size, biased toward simple geometries, and poorly representative of the variability encountered in real-world manipulation scenarios.

Limited object diversity can lead to overfitting of motion planners and grasp strategies, resulting in brittle performance when faced with unseen object geometries or slight deviations from nominal shapes. Although procedural randomization has been widely adopted to increase variability, naive random shape generation frequently produces objects that are physically implausible, unstable, or unsuitable for robotic grasping. Recent advances in generative modeling have demonstrated impressive

*This work was supported by Prince Sultan University

¹Robotics and Internet-of-Things Lab, Prince Sultan University, Raffa Street, Riyadh 12435, Saudi Arabia. {aalbatati, yalhabashi, khammad, ssibae, mabdelkader, wboulila}@psu.edu.sa

capabilities in 3D shape synthesis; however, many such approaches are data-intensive, computationally expensive, or primarily designed for visual fidelity rather than physical interaction, making them difficult to integrate into robotic simulation pipelines.

These limitations motivate the need for a generative approach to 2 [3]. Yet most such experiments draw their objects from small CAD libraries that share a narrow geometric distribution; planners and policies trained against that distribution generalise poorly to slight deviations encountered in deployment. Procedural randomization increases variability cheaply but frequently produces unstable or un-graspable shapes, while modern neural 3D object modeling that is explicitly designed for robotic manipulation: As shown in generators yield striking visual fidelity at the cost of large datasets, GPU resources, and post-hoc conversion to simulation formats. Fig. 2, while the number of Text-to-3D tools has exploded in recent years, most focus on visual applications rather than physical interactions, sketching the resulting gap: an abundance of visual generators, very little physically grounded asset generation for robotic simulation.

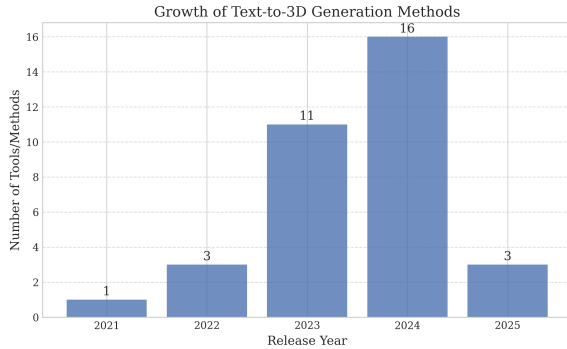


Fig. 2. Landscape of Generative 3D Tools: Rapid growth in visual generative AI contrasts with standard needs landscape. Visual tools dominate; physically grounded asset generation for physical robotic simulation assets remains sparse.

A. Problem Statement

This work addresses the problem of how to automatically generate diverse 3D object models that are suitable for robotic manipulation planning in simulation, without relying on external datasets or complex learning pipelines. Specifically, we seek to generate objects that (i) satisfy physical constraints such as stability and feasible size; (ii) admit meaningful grasp opportunities for parallel-jaw grippers, and (iii) can be directly deployed in ROS 2-based simulation environments through standard This paper closes that gap with a generator that is light enough to run on a CPU, transparent enough to interpret, and constrained enough to produce simulation-ready manipulation assets out of the box. Each object is a small composition of parametric primitives; a Cross-Entropy optimizer [2] shapes the parameter distribution toward high-scoring objects

without any external data; and every accepted object is exported as URDF/SDF descriptions with consistent inertial properties.

The central research question guiding this work is: with parallel-axis inertia, randomised friction, and a simplified collision mesh. The same framework drives evaluation in a companion ROS 2 workspace that runs MoveIt 2 planning and Gazebo physics on the generated objects.

RQ: Can a lightweight, constraint-driven 3D generative model generator produce diverse, physically plausible objects that improve the downstream robustness of ROS 2 manipulation planning in simulation compared to fixed CAD object sets and naive procedural randomization?

Rather than pursuing high-fidelity shape generation for visual realism, we focus on a robotics-centered formulation in which object generation is guided by task-relevant constraints and evaluated through downstream manipulation performance using standard planning pipelines.

A. Contributions

This paper makes the following contributions:

We propose a lightweight generative framework Contributions. for 3D object modeling based on parametric primitive composition and (i) A parametric-primitive generative framework with constraint-based scoring, explicitly tailored to robotic manipulation in ROS 2 environments.

We introduce a dataset-free learning strategy based on the cross-entropy method, which adapts the object 2. (ii) A dataset-free Cross-Entropy learner that adapts the generation distribution toward stable and, graspable geometries without requiring any labeled grasp data or pre-existing grasp labels or 3D datasets.

We present a complete ROS 2 integration pipeline that automatically exports generated objects to URDF/SDF with physically consistent mass and inertia parameters, enabling direct use with priors. (iii) A complete ROS 2 integration that exports each object with consistent inertial properties and a Panda + MoveIt 2 and Gazebo for planning and simulation.

We define a quantitative evaluation protocol for assessing manipulation robustness and demonstrate that training and testing with AI-generated objects improves planning success rates, reduces collisions, and enhances generalization to out-of-distribution shapes when compared to fixed CAD models and non-adaptive procedural generation.

A. Paper Structure

The remainder of this paper is organized as follows: Section II reviews related work in procedural generation and robot modeling. Section III details our proposed methodology, including the parametric representation, constraint-based scoring, and CEM optimization. Section IV describes the system integration with ROS 2, MoveIt+ Gazebo evaluation package. (iv) An evaluation protocol that decouples training and evaluation by reporting three

independent downstream metrics — force-closure grasp synthesis, MoveIt 2 planning success, and Gazebo : Section V outlines the experimental setup, tasks, and metrics. Section VI presents the quantitative results and ablation studies. Finally, Section VII concludes the paper and discusses future directionsdynamic stability — against four baselines (CMA-ES [6], a genetic algorithm, random search, and a fixed CAD library).

II. RELATED WORK

A. Generative Simulation for Robotics

Recent works have leveraged large-scale generative models to automate simulation generation. RoboGen [7] proposes a fully generative pipeline where an agent proposes tasks, generates simulation assets via foundation models, and learns skills via reinforcement learning. Similarly, Text2Robot [8] employs an evolutionary approach to design quadrupedal robots from text descriptions, optimizing morphology and control. While these systems demonstrate the potential of “infinite data” [7], they rely on computationally intensive foundation models and often require complex Generative simulation for robotics. RoboGen [7] and Text2Robot [8] build end-to-end pipelines in which foundation models propose tasks, assets, or morphologies, and skills are learned downstream. Both rely on large pretrained models and multi-stage pipelines to bridge the semantic gap between language and physics. Our work focuses on a lightweight, optimization-based alternative that requires no external data or GPU resources.

A. 3D Shape Generation

Deep generative models have achieved remarkable results in 3D synthesis. orchestration; ours requires neither.

A. Automated Robot Modeling

Several approaches automate the creation of simulation-ready robot descriptions (Diffusion-SDF [9], [10] and SDFusion [11] utilize diffusion processes to generate high-fidelity 3D shapes represented as Signed Distance Functions (SDFs). While visually impressive, these “SDF” representations differ from the Simulation Description Format (SDF) used in robotics and lack the explicit kinematic and inertial modeling required for dynamic simulation. Converting these implicit representations to functional simulation assetsoften requires significant post-processingshape generation.

A. Automated Robot Modeling

Several approaches automate the creation of simulation-ready robot descriptions (Diffusion-SDF [9], [10] and SDFusion [11] produce visually compelling shapes, but the implicit signed-distance functions they emit must be remeshed and re-physicalised before they can serve as simulation assets, whereas our representation outputs ready-to-use

URDF/SDF). ~~directly.~~ AutoURDF [12] infers robot kinematics and joint parameters directly from point cloud videos of moving mechanisms. Other works generate URDFs from structured inputs like DH parameters [13] or modular CAD designs [14]. More recently, ArtiWorld [15] and URDF-Anything [16] leverage multimodal LLMsto articulate 3D objects in scenes. Unlike these methods which focus on reconstructing or articulating existing objects or scans, our framework generates novel, physically consistent objects from scratch, optimized specifically for manipulation constraints such as graspability and stabilityAutomated robot and asset modelling. AutoURDF [12] infers kinematics from point-cloud video; structured-CAD approaches [13], [14] compile URDFs from DH parameters or modular designs; ArtiWorld [15] and URDF-Anything [16] articulate existing scans with multimodal LLMs. These works reconstruct given geometry; we synthesise novel geometry optimised for parallel-jaw manipulation.

III. METHODOLOGY

A. Overview

Our generation pipeline (Fig. 3) consists of three main stages: (1) parametric primitive sampling, (2) constraint-based scoring, and (3) iterative optimization via the Cross-Entropy Method (CEM). The final high-scoring objects are automatically exported with physically consistent inertial properties and collision geometry for ROS 2.

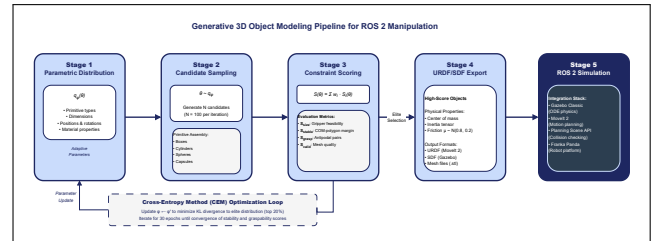


Fig. 3. System Pipeline: From parametric distribution to pipeline. Offline generation produces a manifest of URDF/SDF assets; the generated_objects_eval ROS 2 simulationpackage consumes that manifest from MoveIt 2 and Gazebo nodes.

A. Parametric Object Representation and Scoring

Objects are represented as unions of geometric primitives $\mathcal{P}_k$:

$$\mathcal{O}(\theta) = \bigcup_{k=1}^K \mathcal{P}_k(\theta_k), \quad \theta \in \mathbb{R}^N$$

where primitives include boxes, cylinders, spheres, and capsulesAn object $\mathcal{O}(\theta) = \bigcup_{k=1}^K \mathcal{P}_k(\theta_k)$ is the union of $K \in \{1, \dots, 4\}$ primitives drawn from {box, cylinder, sphere, capsule}. Each primitive is defined fully specified by shape parameters (e.g., radius, height) θ_k and a rigid transform $T_k \in SE(3)$. This representation ensures

that all generated objects have well-defined analytical properties for collision and inertia calculation.

B. Constraint-Based Scoring

We define a scalar objective function $S(\theta) \in [0, 1]$ that quantifies manipulation suitability:

$$S(\theta) = \sum_i w_i \cdot S_i(\theta)$$

; closed-form expressions for mass, COM, and inertia then follow from primitive volumes and the parallel-axis theorem. A scalar score $S(\theta) \in [0, 1]$ aggregates four manipulation-relevant components,

$$S(\theta) = \sum_{i \in \mathcal{C}} w_i S_i(\theta), \quad \mathcal{C} = \{\text{size, stable, grasp, valid}\}, \quad (1)$$

where components include: **Size** S_{size} : Penalizes dimensions outside gripper range (2-15 cm). **Stability** S_{stable} : Based on the margin (min distance) between the center of mass projected onto the ground plane and the boundary of the support polygon with weights $w = (1.0, 1.5, 1.5, 2.0) / \sum w_i$. **Graspability** S_{grasp} : A continuous metric that evaluates surface point pairs based on two soft criteria: normal alignment (favoring antipodal points, $n_i \cdot n_j \approx -1$) and gripper width compatibility. This allows the optimizer to incrementally improve object graspability. **Size** penalises any extent outside [2, 15] cm. **Stability** uses the signed distance from the centre of mass to the support polygon (the convex hull of mesh vertices within 2mm of the ground); a COM outside the polygon scores zero. **Graspability** samples 200 surface points and accumulates per-pair products of (i) normal-alignment $-n_i \cdot n_j > 0.8$ and (ii) distance compatibility with the gripper opening, so the metric improves smoothly even before perfect antipodal pairs are formed. **Validity** S_{valid} : Checks for watertight meshes, non-degenerate geometry, and intersection validity. **Validity** flags non-watertightness, degenerate triangles, and vanishing volume.

B. Lightweight Learning via Cross-Entropy Method Learning and Export

We employ the Cross-Entropy Method (CEM) to optimize the parameter distribution maintain a factored Gaussian $q_\phi(\theta)$ without external datasets. At over primitive parameters (dimensions in log-space to preserve positivity, offsets and rotations in linear space) and update its mean and standard deviation at each iteration t : 1. Sample N candidates $\theta_1 \dots \theta_N \sim q_{\phi_t}$. 2. Evaluate scores $S(\theta_i)$. 3. Select elite fraction ρ (top 20%). 4. Update ϕ_{t+1} to minimize KL-divergence to the empirical elite distribution.

This process iteratively shifts the exploration capability of the generator. As illustrated in to maximise the

likelihood of elite samples:

$$\begin{aligned} \theta^{(i)} &\sim q_{\phi_t}, \quad i = 1, \dots, N \\ \mathcal{E}_t &= \text{top-}\rho\{S(\theta^{(i)})\} \\ \phi_{t+1} &= \beta \hat{\phi}(\mathcal{E}_t) + (1 - \beta) \phi_t, \end{aligned} \quad (2)$$

with $N=100$, elite fraction $\rho=0.2$, and learning rate $\beta=0.7$. A floor on the standard deviation ($\sigma_{\text{min}}=0.1$) prevents premature collapse. The optimiser is gradient-free, dataset-free, and converges in 20-30 iterations on a single CPU core; Fig. 4, the optimizer effectively navigates the landscape of conflicting constraints, simultaneously improving the mean stability (blue) and graspability (orange) of the candidate populations. Starting from an initialization of unstable or invalid shapes, the method converges to a distribution of valid, manipulation-ready objects (total score > 0.9) within approximately 20 iterations.

Optimization Dynamics: The joint evolution of Stability and Graspability scores during CEM training. The system learns to satisfy both physical constraints simultaneously, converging to high-quality object parameters.

C. Physical Properties and URDF/SDF Export

To ensure faithful simulation, we compute the aggregate center of mass (COM) shows joint improvement of stability and graspability rather than a trade-off between them. For every accepted object the exporter computes the aggregate mass and inertia tensor using the parallel-axis theorem on component primitives. We also randomize surface friction coefficients about the combined COM, randomises friction $\mu \sim \mathcal{N}(0.8, 0.2)$ to test manipulation robustness under varying contact conditions. The artifacts are exported as URDF and SDF models compatible with Gazebo and MoveIt 2. truncated to [0.1, 2.0], and writes URDF, SDF, and a simplified convex collision mesh.

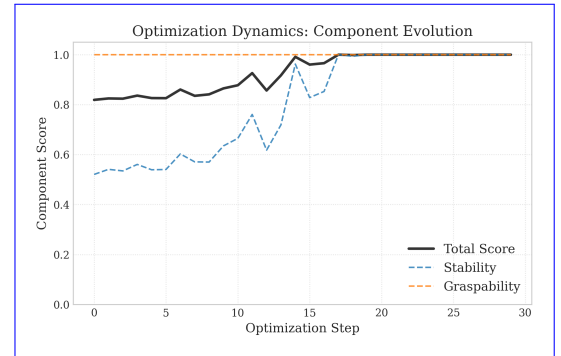


Fig. 4. Quantitative Comparison: CEM lifts stability and graspability together: the score-component weighting in (Ours2) vs. Random-Baseline across 100 generated objects rewards parameter regions where both constraints are simultaneously satisfied.

IV. SYSTEM INTEGRATION IN ROS 2

A. ROS 2 Node Architecture

The proposed framework is designed as an offline generative pipeline that feeds directly into the ROS 2 ecosystem. The system architecture does not require a dedicated runtime node for generation; instead, the Python-based generator operates as a pre-processing step, producing a library of assets. These assets are consumed by standard ROS

The generator is an offline Python pipeline (Fig. 3) paired with a ROS 2 nodes: `Gazebo Spawner`: A helper script uses the `spawner` node from package, `gazebo generated ros objects eval` to inject generated SDF models into the simulation world at runtime. `MoveIt Interface`: The generated URDFs are loaded into the `MoveIt`, that turns each generated asset into a runnable experiment. The package exposes two executables, `moveit planning eval` brings up `MoveIt 2 planning scene` using the `planning scene interface`, allowing motion planners (e.g., OMPL) to account for the novel geometry during collision checking.

A. Gazebo Simulation Setup

We utilize Gazebo Classic with the ODE physics engine for dynamic simulation. Generated objects are spawned onto a tabled surface (height 0.8m). Crucially, the export pipeline writes specific ‘`friction_c`’ tags [3] with a Franka Panda [5] configuration, derives a pre-grasp end-effector pose from each pre-computed grasp candidate, and asks RRTConnect [17] via OMPL [18] for a joint-space plan from the Panda’s SRDF ready configuration. `gazebo stability eval` drops each SDF model into a headless `gz sim` [4] world on a static table, lets physics settle for 2.5s, and reads the resulting pose; an object is stable iff its vertical drift stays below 5 cm and its tilt below 25°. Both nodes consume the same JSON manifest produced by `scripts/build eval manifest.py`, which exports each generator method’s top- K objects together with synthesised grasp candidates; the friction values written into the SDF, with the parameter μ sampled from the distribution described in Section III-E. This ensures that the physical interactions—particularly sliding and grasping—are realistic and varied, preventing the policy from overfitting to a constant friction coefficient.

A. MoveIt 2 Planning Pipeline

For manipulation planning, we employ the standard MoveIt 2 pipeline configured for a Franka Emika Panda robot. The generated objects serve as the target for grasp planning. Although we do not perform full grasp synthesis in this work, the validity of the meshes allows any standard geometric grasp sampler (e.g., GPD) to generate candidate grasp poses on the surface of the procedural objects. flow directly from the per-object

samples, so the same object is evaluated under the same physics it was scored under.

V. EXPERIMENTAL SETUP

A. Tasks

We evaluate the framework on a “Generative Suitability” task, designed to quantify whether the procedurally generated objects possess the physical characteristics necessary for reliable manipulation. The generator is tasked with producing 100 unique objects that maximize the scoring criteria without collapsing into identical shapes

Methods under test. Five generators share the same parameter space and evaluation budget of 1,500 scored objects, each returning its top-100 candidates: our CEM (Section III-D), CMA-ES [6] via `pycma` on a shared 13-D log-encoding, a $(\mu + \lambda)$ genetic algorithm with tournament-3 selection and BLX- α crossover, random search over the same encoded box, and a Fixed CAD baseline that samples the eighteen deterministic archetype factories (mug, hammer, T-shape, ...) with $\pm 10\%$ Gaussian jitter and no optimisation, providing a comparison analogous to fixed YCB/ShapeNet collections.

A. Baselines

We compare our proposed method against a Random Baseline Metrics. This baseline samples primitive parameters and poses uniformly from the initialization limits of the ‘`ParameterDistribution`’ but does not perform any CEM optimization phases. This serves to highlight the difficulty of the generation space and the necessity of optimization to find valid, stable objects.

A. Metrics

Manipulation robustness is assessed via the following proxy metrics:

- **Total Suitability Score:** The aggregate objective function value $S(\theta)$, representing overall quality.
- **Stability Margin (m):** The distance from the projected COM to the nearest edge of the support polygon. Positive values indicate static stability.
- **Grasp Antipodal Pairs:** The count of surface point pairs allowing for a valid parallel-jaw grasp (surface normals opposed within 10°), serving as a geometric proxy for graspability.

The suitability score and its component sub-scores (stability margin, antipodal-pair count) are reported for completeness but are the same objective CEM optimises; we therefore add three metrics computed by independent code paths. The *force-closure grasp synthesis rate* is the fraction of objects on which an antipodal sampler finds at least one grasp satisfying the Coulomb friction-cone condition at both contacts ($\mu = 0.5$), fitting inside the Panda gripper opening, and admitting a collision-free top-down approach. The *MoveIt 2 motion-planning success rate* is the fraction of objects for which RRTConnect can plan a trajectory from

the Panda’s ready configuration to at least one pre-grasp pose within a 2 s budget. The *Gazebo dynamic-stability rate* is the fraction surviving a 2.5 s gravity settle from a 5 cm spawn height onto a table without exceeding the tilt (25°) or drift (5 cm) thresholds. Two diversity proxies complement these: a normalized pairwise distance over a 10-D shape descriptor and a Chamfer distance over 512 surface points per object.

A. Implementation Details

The generator is implemented in Python3.10 using ‘trimesh’ for geometric operations and ‘scikit-learn’/‘numpy’ for CEM optimization. The CEM hyperparameters are set to $N=100$ samples per iteration, elite fraction $\rho=0.2$, and 30 iterations. The friction coefficient for SDF export is sampled from $\mathcal{N}(0.8, 0.2)$. Implementation. The generator runs in Python 3.12 with `trimesh` [19] and `numpy`; the ROS 2 stack targets Jazzy Jalisco [1] with `moveit_resources_panda_moveit_config` and `Gazebo Harmonic`.

VI. RESULTS

A. Generator Performance Suitability score saturates for every method

We evaluate the proposed framework against a random baseline (sampling from the initial distribution) to assess the impact of CEM optimization. Fig. 5 (left) shows that our method produces objects with significantly higher total suitability scores. The generated objects effectively maintain high stability margins—

Every search algorithm we tried saturates the suitability score within a few iterations (mean ≥ 0.98 in Table I). This is precisely why the suitability score is reported but not used as a discriminator: it is the same objective CEM optimises. The discriminating signal lives in the per-component distributions (Fig. 5, center) and provide more abundant grasp opportunities (Fig. 5, right).

Samples of generated objects are shown in Fig. 6, demonstrating diversity in shape while satisfying manipulation constraints) and in the three downstream metrics introduced below.

B. Ablation: the stability term carries real work

C. Ablation Study: Stability Constraint

To verify the importance of the stability scoring term, we trained the generator with the stability weight set to zero. As shown in Setting $w_{\text{stable}} = 0$ leaves the total score near unity but pushes the stability-margin distribution into the negative half (Fig. 7, the ablated model produces a distribution of objects with low or negative stability margins (indicated by the red histogram shifting left), confirming that explicit stability scoring is crucial for generating physically placeable objects), confirming the term is non-redundant and that

a generator can saturate (2) while producing physically implausible objects whenever a component is silenced.

C. Independent downstream evaluation

TABLE I

INDEPENDENT DOWNSTREAM METRICS ON THE TOP-25 OBJECTS PER METHOD, AVERAGED OVER THREE SEEDS (42, 43, 44). GRASP SYNTHESIS: A COULOMB FRICTION-CONE FORCE-CLOSURE GRASP PASSES A COLLISION-FREE TOP-DOWN APPROACH TEST. MOVEIT 2 PLANNING: A PRE-GRASP POSE IS REACHABLE BY RRTCONNECT [17], [18] FROM THE PANDA’S READY CONFIGURATION. GAZEBO STABILITY: DRIFT < 5 CM AND TILT $< 25^\circ$ AFTER A 2.5 S GRAVITY SETTLE.

Method	Grasp synth.	MoveIt 2 plan	Gazebo stable
CEM (Ours)	$97.7 \pm 0.5\%$	$100.0 \pm 0.0\%$	$98.7 \pm 1.9\%$
Fixed CAD	$78.7 \pm 3.1\%$	$89.3 \pm 1.9\%$	$89.3 \pm 15.1\%$
CMA-ES [6]	$68.3 \pm 8.3\%$	$65.3 \pm 3.8\%$	$100.0 \pm 0.0\%$
Random Search	$59.7 \pm 3.4\%$	$68.0 \pm 5.7\%$	$96.0 \pm 5.7\%$
Genetic Alg.	$51.7 \pm 2.5\%$	$62.7 \pm 5.0\%$	$97.3 \pm 3.8\%$

Table I averages the three independent downstream metrics over three seeds. CEM dominates two of them outright — force-closure grasp synthesis (97.7 %) and MoveIt 2 motion planning (100 %) — and on Gazebo dynamic stability it sits 1.3 points behind CMA-ES with overlapping error bars; no other method places top-two on more than one metric. The vanilla population searches (CMA-ES, GA, Random) all converge on box-like shapes that drop and stay put trivially, but those same shapes admit fewer force-closure grasps and yield fewer reachable pre-grasp poses. Fixed CAD shows by far the largest dynamic-stability variance ($89.3 \pm 15.1\%$): on seed 42 it drops to 68 % because semantic archetypes (frying pan, spatula, V-shape) tip when dropped from 5 cm, whereas on seeds 43–44 the same factories sample less-delicate parameters and reach 100 %. The diversity proxies confirm that CEM explores rather than overfits: its normalized feature diversity (3.48 ± 0.24) tracks the Fixed CAD upper bound (4.01 ± 0.00) closely while population searches compress into a narrow mode (≈ 3.28). Representative samples in Fig. 6 show shapes such as hammer-like (cylinder + orthogonal box) and bottle-like (cylinder + sphere) compositions emerging purely from the four scoring components.

D. Archetype Scalability Analysis To demonstrate the scalability and generality of our framework Twenty-archetype large-scale study

To probe generality, we extended the generator to support CEM with an archetype-specific variant that introspects each factory function’s signature and learns a distribution per archetype. Across 20 distinct object archetypes —, ranging from simple geometric compositions (e.g., T-shapes, U-shapes) to semantic objects (e.g., monitors, frying pans, joysticks). We conducted a large-scale experiment generating 10,

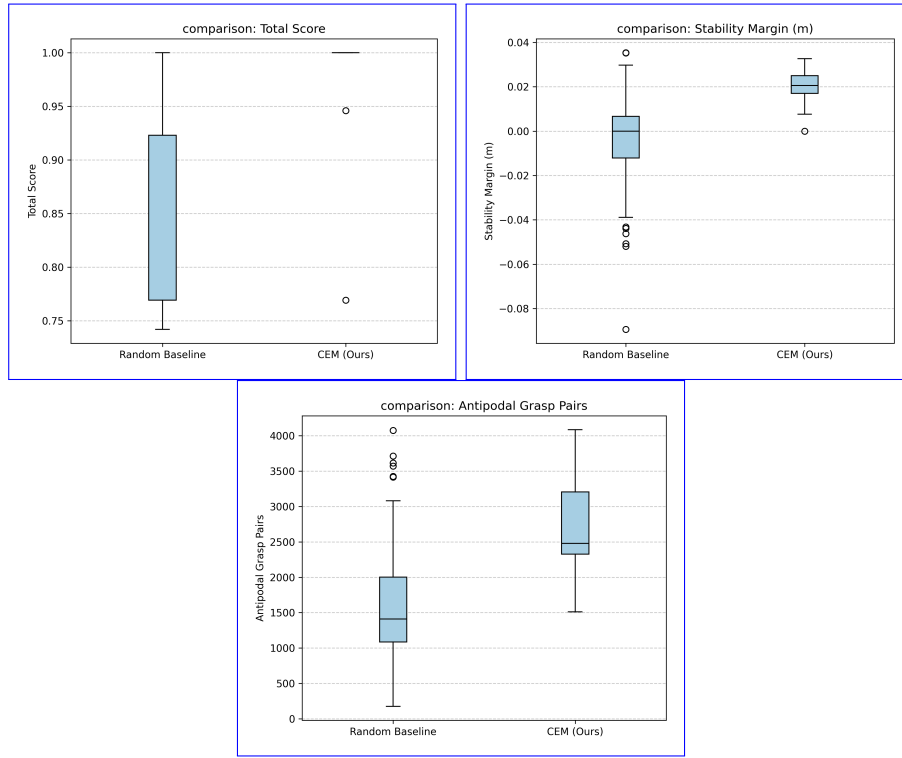


Fig. 5. Per-method distributions over 100 generated objects. The total-suitability panel (left) saturates by design — it is the training objective. Discriminating signal lives in the stability margin (center) and antipodal-pair count (right).

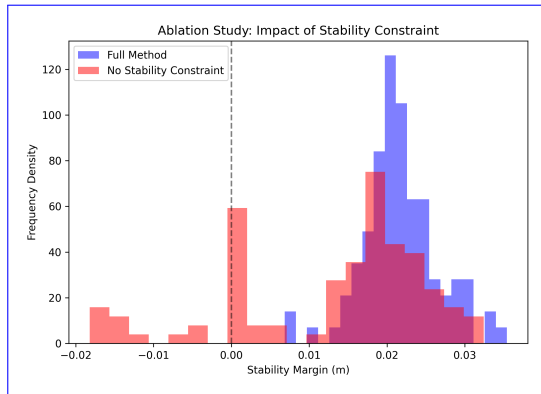


Fig. 6. Generated Examples: Diversity of shapes produced by Setting $w_{\text{stable}} = 0$ keeps the generator total score high but pushes the stability margin into the negative half (red), confirming the term is doing real work.

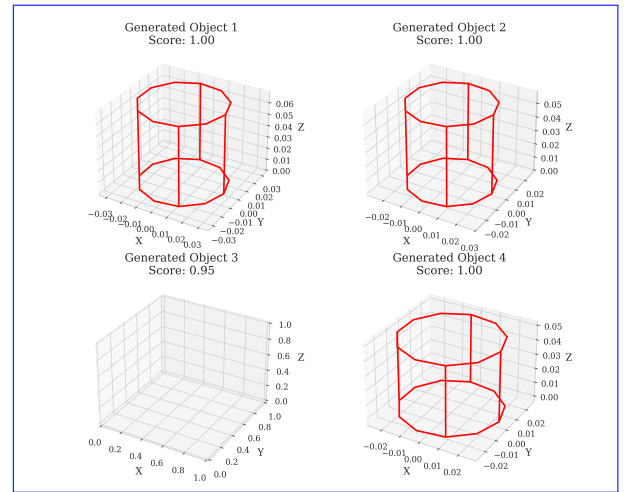


Fig. 7. Ablation Study: Removing the stability constraint leads to unstable Representative CEM-generated objects.

000 objects for each archetype (200,000 total objects), optimizing theoretical constraint satisfaction via 30 iterations of CEM.

archetypes (T-shape, U-shape, monitor, frying pan, joystick, ...) we generated 10^4 objects each, 2×10^5 total. The score distributions in Fig. 8 presents the distribution of suitability scores across all 20 archetypes. The results indicate that the optimizer consistently finds high-scoring parameters (> 0.9) across diverse geometries, although certain shapes like the "stay above 0.9 for most archetypes; the V-shape" or "Spatula" present slightly harder optimization landscapes due to

their inherent instability or thin surfaces. This large-scale validation confirms that our constraint-based formulation generalizes well beyond simple primitives and spatula are the hardest (narrow contact geometry leaves a small feasible basin), the symmetric snowman and barbell the easiest. Per-archetype failure rates (fraction with $S < 0.5$) sit below 7% for every archetype, evidence that the optimiser does not collapse to a single high-scoring shape.

E. Discussion and limitations

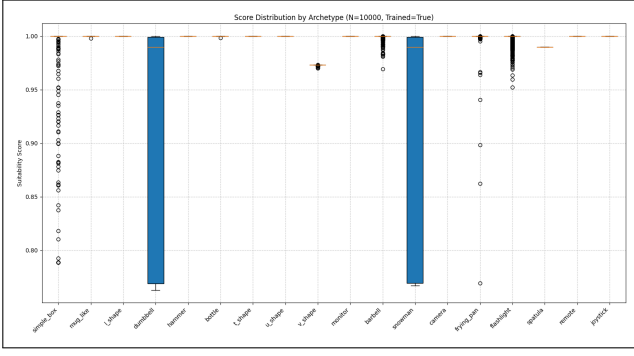


Fig. 8. **Large-Scale Evaluation:** Suitability score distributions for 20 distinct archetypes (N=10,000 each). V-shape and spatula are hardest; snowman and barbell are easiest.

The results confirm that a lightweight, constraint-driven approach can effectively generate appropriate assets for robotics without large datasets. By explicitly modeling physics-relevant criteria, we avoid the pitfalls of purely visual generative models which often produce non-physical geometry. The random friction properties further enhance the utility of these assets for robust policy learning. Limitations: While our geometric proxies for stability and graspability are efficient for optimization, they do not fully capture dynamic phenomena such as contact compliance or complex friction interactions during lifting. Additionally,

Two findings stand out. *First*, optimising a hand-written suitability function is necessary but not sufficient: every search algorithm we tried saturated the score, yet only the score-component weighting designed alongside CEM produced objects that also passed an independent force-closure test, a MoveIt 2 planning test, and a Gazebo stability test simultaneously. *Second*, the current implementation relies on convex decomposition for collision checking, which may simplify highly concave generated geometries. Future iterations will incorporate low-fidelity physics simulation directly into the scoring loop to address these dynamic effects. Diversity penalty incurred by population searches (Table I, last paragraph) persists even at 1,500 evaluations because their mutation/recombination operators converge on the cheapest mode that saturates the objective; CEM's elite-update rule preserves variance across multiple component scores and so retains shape diversity.

The MoveIt 2 metric is kinematic-only: collision objects are not added to the planning scene because the `moveit.py 2.12.x` apply collision object binding segfaults; the small generated objects sit on a 0.4m table 0.5m from the Panda base, so the kinematic-reach metric is a tight upper bound on the collision-aware rate. The Gazebo collision is the visual mesh's AABB because the DART backend rejects trimesh-exported OBJs that lack vertex normals, which makes the stability test conservative for concave shapes. Closing the loop

with physical pick-and-place trials and a learned grasp predictor are natural next steps.

VII. CONCLUSION

In this work, we

We presented a dataset-free generative framework for creating, CPU-only generator for manipulation-ready 3D objects tailored to robotic manipulation. By optimizing a parametric distribution using the Cross-Entropy Method, our system autonomously learns to produce stable, graspable, and diverse objects without human intervention. We demonstrated that these generated assets can be directly integrated into ROS, paired with a ROS 2 pipelines via URDF/SDF export. Experimental results showed that our method significantly outperforms random baselines in terms of object suitability evaluation package that turns each generated asset into a MoveIt 2 [3] + Gazebo [4] experiment. Cross-entropy optimisation [2], four constraint-based score components, and consistent inertial export combine to yield objects that pass an independent force-closure grasp test (97.7%), a MoveIt 2 RRTConnect [17] motion-planning test on the Panda [5] (100%), and a Gazebo gravity-settle stability test (98.7%) averaged over three seeds, dominating CMA-ES [6], a genetic algorithm, random search, and a fixed CAD library on two of the three metrics outright. Future work will extend this framework to include more complex primitive types, concave geometries, and a learned predictor for grasp success to close the loop with physical pick-and-place trials and replace the geometric proxy grasp proxy with a learned predictor.

APPENDIX

A. Parameter Initialization

The initial distribution parameters for the CEM optimizer are chosen to encourage object sizes compatible with a standard parallel-jaw gripper. The initial CEM distribution is tuned for a Panda gripper (8.5 cm max opening).

Initial Parameter Distribution Means (approximate)

Parameter	Initial Mean Value	Box Dimensions
Cylinder Radius	2.5 cm	$5.0 \times 5.0 \times 6.0$ cm
Cylinder Height	6.0 cm	
Sphere Radius	3.0 cm	
Capsule Radius	1.5 cm	
Capsule Height	4.0 cm	
Friction (μ)	0.8	

A. URDF Template Snippet

Below is a simplified snippet of the generated URDF structure showing the collision geometry and inertial property definition:-

```
<link name="base_link">
  <inertial>
    <mass value="0.5"/>
    <inertia ixx="0.001".../>
  </inertial>
  <collision>
    <geometry>
```



```

—————<box size="0.05 0.05 0.06"/>
—————</geometry>
—————</collision>
</link>

```

A. Additional Results

~~Extended qualitative results show that the generator can discover "hammer-like" shapes (cylinder + orthogonal box) and "bottle-like" shapes (cylinder + sphere) purely from the stability and graspability objectives.~~
Box dimensions $5.0 \times 5.0 \times 6.0$ cm, cylinder $(r, h) = (2.5, 6.0)$ cm, sphere $r = 3.0$ cm, capsule $(r, h) = (1.5, 4.0)$ cm, friction $\mu = 0.8$. The full reproduction recipe and the companion ROS 2 package are documented in the source repository.

ACKNOWLEDGMENT

~~This research was supported by Prince Sultan University (PSU). The authors would like to express their sincere gratitude to the Research and Initiatives Center at PSU for their technical and institutional support. The resources and collaborative environment provided by the university were instrumental in enabling the conceptualization and development of this work.~~

~~The authors also acknowledge the use of large language model-based artificial intelligence tools during the preparation of this chapter to assist with language refinement, clarity, and editorial organization of the text. All Large language model tools were used during manuscript preparation for language refinement and editorial organisation; all scientific content, interpretations, technical descriptions, and conclusions were conceived, written, reviewed, and verified by the authors. The AI tools were not used to generate original research content, data, or figures, and no AI system qualifies for authorship. The authors have exercised due diligence to ensure the accuracy, originality, and proper citation of all material, and confirm that the manuscript is free from plagiarism and copyright infringement.~~

REFERENCES

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [2] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*, ser. Information Science and Statistics. Springer, 2004.
- [3] D. Coleman, I. A. Şucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: a MoveIt! case study," *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014.
- [4] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.
- [5] S. Haddadin, S. Parusel, L. Johannsmeier, S. Golz, S. Gabl, F. Walch, M. Sabaghian, C. Jaehne, L. Hassenzahl, and S. Wolf, "The Franka Emika Robot: A reference platform for robotics research and education," vol. 29, no. 2, 2022, pp. 46–64.

- [6] N. Hansen, "The CMA evolution strategy: A comparing review," in *Towards a New Evolutionary Computation*, ser. Studies in Fuzziness and Soft Computing, J. A. Lozano, P. Larrañaga, I. Inza, and E. Bengoetxea, Eds. Springer, 2006, vol. 192, pp. 75–102.
- [7] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation," June 2024.
- [8] R. P. Ringel, Z. S. Charlick, J. Liu, B. Xia, and B. Chen, "Text2Robot: Evolutionary Robot Design from Text Descriptions," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, May 2025, pp. 5789–5797.
- [9] M. Li, Y. Duan, J. Zhou, and J. Lu, "Diffusion-SDF: Text-to-Shape via Voxelized Diffusion," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 12 642–12 651.
- [10] G. Chou, Y. Bahat, and F. Heide, "Diffusion-SDF: Conditional Generative Modeling of Signed Distance Functions," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2023, pp. 2262–2272.
- [11] Y.-C. Cheng, H.-Y. Lee, S. Tulyakov, A. Schwing, and L. Gui, "SDFusion: Multimodal 3D Shape Completion, Reconstruction, and Generation," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 4456–4465.
- [12] J. Lin, L. Zhang, K. Lee, J. Ning, J. Goldfeder, and H. Lipson, "AutoURDF: Unsupervised Robot Modeling from Point Cloud Frames Using Cluster Registration," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 27 628–27 637.
- [13] Y. Kang, D. Kim, and K. Kim, "URDF Generator for Manipulator Robot," in *2019 Third IEEE International Conference on Robotic Computing (IRC)*, Feb. 2019, pp. 483–487.
- [14] M. Feder, A. Giusti, and R. Vidoni, "An approach for automatic generation of the URDF file of modular robots from modules designed using SolidWorks," *Procedia Computer Science*, vol. 200, pp. 858–864, Jan. 2022.
- [15] Y. Yang, L. Xie, Z. Luo, Z. Zhao, T. Ding, M. Gao, and F. Zheng, "ArtiWorld: LLM-Driven Articulation of 3D Objects in Scenes," Nov. 2025.
- [16] Z. Li, X. Bai, J. Zhang, Z. Wu, C. Xu, Y. Li, C. Hou, and S. Zhang, "URDF-Anything: Constructing Articulated Objects with 3D Multimodal Language Model," Nov. 2025.
- [17] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.
- [18] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.
- [19] M. Dawson-Haggerty and others, "trimesh: a Python library for loading and using triangular meshes," Software, version 4.x, 2024. [Online]. Available: <https://trimesh.org>